Full-Stack Lab Test Revision: Spring Boot, MongoDB, Vue, Servlets

Oct 11, 2026 · @Hemnath D

Start here: the basics from zero

Read this section first if you have forgotten the fundamentals. Everything later builds on it.

1. How a web application works

Think of a restaurant. The **frontend** (Vue) is the dining room where the customer sees the menu and orders. The **backend** (Spring Boot or a servlet) is the kitchen that receives orders and prepares them. The **database** (MongoDB or MySQL) is the storeroom where data is kept permanently. The waiter who carries orders between dining room and kitchen is **HTTP**.

```
URL anatomy:   http://localhost:8080/api/students/42?name=ra
               |      |         |    |            |  |
               |      |         |    |            |  query parameter (?
key=value)
               |      |         |    |            path variable (42)
               |      |         |    path (which resource)
               |      |         port (which program on the machine)
               |      host (localhost = your own computer)
               protocol
```

A **request** goes from client to server and contains a method, a URL, headers and sometimes a body. The server returns a **response** with a status code and usually a body.

HTTP method	Meaning	CRUD word
GET	fetch data, changes nothing	Read
POST	send new data	Create
PUT	replace or update existing data	Update
DELETE	remove data	Delete

Status code	Meaning
200 OK	success
201 Created	new item saved
204 No Content	success, nothing to return (used for delete)
400 Bad Request	client sent wrong data
404 Not Found	wrong URL or item does not exist
405 Method Not Allowed	URL exists but not for this method (POST sent, only GET coded)
500 Internal Server Error	bug or exception in your server code

JSON is the text format used to send objects between frontend and backend:

```
{ "name": "Asha", "age": 21, "skills": ["Java", "Vue"] }
```

Keys are in double quotes; values can be text, numbers, true/false, arrays `[ ]` or objects `{ }`. Spring converts a Java object to JSON automatically and back; axios does the same on the Vue side.

Port = a numbered door on your computer. Each program listens on its own: Vue dev server 5173, Spring Boot 8080, Tomcat 8080, MongoDB 27017, MySQL 3306. Two programs cannot use the same port (that causes "port already in use").

2. Tools and what each one does

Tool	What it is	Why you need it
JDK 17+	Java compiler and runtime	to compile and run Java
Maven (`mvn`)	build tool that downloads libraries listed in `pom.xml`	to build and run Spring Boot / servlet projects
IDE (IntelliJ / Eclipse / VS Code)	code editor	to write code
Node.js + npm	JavaScript runtime and package manager	to run Vue tools and install libraries
Vite	fast dev server and bundler used by Vue	gives `npm run dev`
MongoDB	document database	stores data for Part A

Tool	What it is	Why you need it
MySQL	table-based database	stores data for Part B
Tomcat	servlet container	runs servlets in Part B
Postman or curl	API tester	test backend without a frontend

3. Java basics used in this exam

Class and object. A class is a blueprint; an object is one thing built from it.

```
public class Student {            // blueprint
    private String name;          // field (data), private = hidden from
outside
    private int age;

    public Student() {}           // constructor: runs when you write new
Student()

    public String getName() {     // getter: lets outside code read the field
        return name;
    }
    public void setName(String name) {   // setter: lets outside code change
the field
        this.name = name;                 // this = the current object
    }
}

Student s = new Student();        // object
s.setName("Asha");
System.out.println(s.getName());
```

Writing getters, setters and constructors for every class is boring. **Lombok** writes them for you at compile time when you add `@Data`, `@NoArgsConstructor`, `@AllArgsConstructor` above the class. The code you write stays short, but the methods exist.

Interface. A list of method names with no body. In Spring Data you write `interface StudentRepository extends MongoRepository<Student, String>` and Spring creates the working class behind the scenes. You never write `implements`.

Generics `<T>`. The angle brackets say what type a container holds. `List<Student>` is a list of students; `MongoRepository<Student, String>` means "works with Student objects whose id is a String".

Collections. `List<Student> list = new ArrayList<>();` then `list.add(s)`, `list.size()`, `list.get(0)`, and `for (Student x : list) { ... }`.

Optional. A box that may or may not contain a value, used when something might not exist (find by id might find nothing). `opt.isPresent()`, `opt.get()`, `opt.orElse(x)`, `opt.map(...)`.

Lambda `->` and method reference `::`. Short ways to write a small function:

```
repo.findById(id).map(student -> ResponseEntity.ok(student))   // lambda
repo.findById(id).map(ResponseEntity::ok)                      // same thing,
shorter
```

Annotation. A label starting with `@` that tells a framework what to do with the class or method below it. `@GetMapping` = "run this method for GET requests". The framework reads the labels; you do not call them.

Dependency injection. You do not write `new StudentRepository()`. You declare it as a field and Spring supplies a ready-made object:

```
@RestController
@RequiredArgsConstructor           // Lombok builds a constructor for final
fields
public class StudentController {
    private final StudentRepository repo;   // Spring injects it
}
```

The older way is `@Autowired private StudentRepository repo;`. Both are accepted.

Packages and imports. `package com.exam.student.model;` says which folder the file lives in. `import ...;` brings in classes from other packages. If IntelliJ shows red, use Alt+Enter to auto-import.

Exceptions. `try { risky } catch (Exception e) { handle } finally { always runs }`. `try (Connection con = ...) { }` (try-with-resources) closes the resource automatically.

4. JavaScript basics used in Vue

```
const name = 'Asha'            // cannot be reassigned
let count = 0                  // can be reassigned
count = count + 1

const student = { name: 'Asha', age: 21 }       // object
student.name                                    // read a field
const list = [1, 2, 3]                          // array
```

```
function add(a, b) { return a + b }             // normal function
const add2 = (a, b) => a + b                    // arrow function, same thing

const msg = `Hello ${student.name}`             // template string with
backticks
const { name: n, age } = student                // destructuring: pulls
fields out

list.filter(x => x > 1)       // [2, 3]  keep items that pass
list.map(x => x * 2)          // [2, 4, 6]  transform each item
list.find(x => x === 2)       // 2  first match

export default something      // make a file's value available
import something from './file'     // use it in another file
import { ref } from 'vue'          // named import from a library
```

Async and await. Calling the server takes time. A function marked `async` can pause at `await` until the server answers, without freezing the page:

```
async function load() {
  try {
    const res = await axios.get('http://localhost:8080/api/students')
    console.log(res.data)        // the JSON body
  } catch (e) {
    console.log('failed', e.message)
  }
}
```

5. HTML and CSS in two minutes

```
<div class="card" id="box">      <!-- tag, class (reusable label), id (unique
label) -->
  <h2>Title</h2>
  <p>Some text</p>
  <a href="/add">Link</a>
  <button>Click</button>
</div>
```

```
.card { padding: 10px; background: white; }   /* . selects a class */
#box  { margin: 20px; }                       /* # selects an id */
button:hover { background: blue; }            /* style on mouse over */
```

Box model: content, then `padding` (inside space), `border`, `margin` (outside space). `display: flex` places children side by side.

6. The Vue mental model

- A Vue app is a tree of **components**. Each `.vue` file is one component with three parts: `<template>` (what it looks like), `<script setup>` (data and functions), `<style>` (how it looks).
- **Reactive data**: you store values with `ref(...)`. When the value changes, Vue redraws only the parts of the page that use it. That is why you do not manually edit the page.
- **Router** decides which component shows for the current URL.
- **Pinia store** is a shared box of data and functions that any component can open. Use it so the list page and the form page see the same data and the same API functions.
- **axios** sends the HTTP request to Spring Boot.

```
User clicks  ->  component function  ->  store action  ->  axios  ->  Spring
Boot  ->  MongoDB
Page updates <-  store state changes <-  response data <-  JSON   <-  result
```

7. Files you will meet

File	Purpose
`pom.xml`	Maven file: project name, Java version, library list (dependencies)
`application.properties`	Spring settings: port, database address
`package.json`	npm file: project name, library list, scripts (`dev` , `build`)
`node_modules/`	downloaded libraries; never edit, recreated by `npm install`
`index.html`	the only real HTML page of a Vue app
`main.js`	starts Vue
`web.xml`	servlet settings (Part B)
`target/`	Maven output folder; ignore

Part A: Spring Boot + MongoDB + Vue CRUD, the big picture

A CRUD app is three layers talking over HTTP: a Vue frontend calls a Spring Boot REST API, which reads and writes MongoDB. Every example below uses a `Student` entity (id, name, email, department, age); swap the names for whatever your question asks.

```
Browser (Vue 3, port 5173)
  Component  ->  Pinia action  ->  axios call
                                      |  HTTP JSON
                                      v
Spring Boot (port 8080)
  @RestController -> (@Service) -> MongoRepository
                                      |
                                      v
                            MongoDB (port 27017)
```

Build order in the exam (do it in this order, test after each step):

1. Backend: create project, add dependencies, set Mongo URI, write model, repository, controller, enable CORS.
2. Test the API in the browser or Postman before touching Vue (`GET http://localhost:8080/api/students`).
3. Frontend: create Vue project, install `axios pinia vue-router`, write store, views, router.
4. Run both, test add, list, edit, delete.

Folder layout

```
backend/
  pom.xml
  src/main/resources/application.properties
  src/main/java/com/exam/student/
    StudentApplication.java
    model/Student.java
    repository/StudentRepository.java
    controller/StudentController.java

frontend/
  index.html
  vite.config.js
  src/
    main.js
    App.vue
    router/index.js
    stores/studentStore.js
    views/StudentList.vue
    views/StudentForm.vue
    assets/style.css
```

REST mapping (memorise this table)

Operation	HTTP method	URL	Body	Spring annotation
Create	POST	/api/students	JSON student	@PostMapping
Read all	GET	/api/students	none	@GetMapping
Read one	GET	/api/students/{id}	none	@GetMapping("/{id}")
Update	PUT	/api/students/{id}	JSON student	@PutMapping("/{id}")
Delete	DELETE	/api/students/{id}	none	@DeleteMapping("/{id}")

Spring Boot backend

The backend needs five files: `pom.xml`, `application.properties`, the model, the repository and the controller. Generate the project at start.spring.io with **Web, Spring Data MongoDB, Lombok, DevTools**, Java 17+, Maven.

pom.xml (dependencies only)

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-mongodb</artifactId>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

In the IDE, enable **annotation processing** or Lombok getters will show as missing.

application.properties

```
server.port=8080
spring.data.mongodb.uri=mongodb://localhost:27017/studentdb
# equivalent split form:
# spring.data.mongodb.host=localhost
# spring.data.mongodb.port=27017
# spring.data.mongodb.database=studentdb
```

The database and collection are created automatically on the first insert. Property names above are for Spring Boot 3.x; if your lab uses Boot 4, the prefix may be `spring.mongodb.*`, so match your lab's reference project.

Model with Lombok

```
package com.exam.student.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "students")
public class Student {

    @Id
    private String id;          // Mongo generates an ObjectId string
    private String name;
    private String email;
    private String department;
    private int age;
}
```

Lombok annotation	What it generates
@Getter / @Setter	getters / setters for all fields
@ToString	toString()
@EqualsAndHashCode	equals() and hashCode()
@Data	all four above plus @RequiredArgsConstructor
@NoArgsConstructor	empty constructor (needed by Jackson and Mongo)
@AllArgsConstructor	constructor with every field
@Builder	`Student.builder().name("A").age(20).build()`
@Slf4j	a `log` object for logging
@RequiredArgsConstructor	constructor for `final` fields (used for injection)

Repository

```
package com.exam.student.repository;

import com.exam.student.model.Student;
import org.springframework.data.mongodb.repository.MongoRepository;
import java.util.List;
import java.util.Optional;

public interface StudentRepository extends MongoRepository<Student, String> {

    List<Student> findByDepartment(String department);
    Optional<Student> findByEmail(String email);
    List<Student> findByNameContainingIgnoreCase(String name);
    List<Student> findByAgeGreaterThan(int age);
}
```

No implementation is written: Spring generates it. Methods inherited from `MongoRepository`: `save()`, `saveAll()`, `findAll()`, `findById()`, `existsById()`, `count()`, `deleteById()`, `delete()`, `deleteAll()`.

Controller (the main exam file)

```
package com.exam.student.controller;

import com.exam.student.model.Student;
import com.exam.student.repository.StudentRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/students")
@CrossOrigin(origins = "http://localhost:5173")
@RequiredArgsConstructor
public class StudentController {

    private final StudentRepository repo;      // injected by constructor

    // CREATE
    @PostMapping
    public ResponseEntity<Student> create(@RequestBody Student student) {
        Student saved = repo.save(student);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    // READ ALL
    @GetMapping
    public List<Student> getAll() {
        return repo.findAll();
    }

    // READ ONE
    @GetMapping("/{id}")
    public ResponseEntity<Student> getOne(@PathVariable String id) {
        return repo.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    // UPDATE
    @PutMapping("/{id}")
    public ResponseEntity<Student> update(@PathVariable String id,
                                          @RequestBody Student s) {
```

```
        return repo.findById(id).map(existing -> {
            existing.setName(s.getName());
            existing.setEmail(s.getEmail());
            existing.setDepartment(s.getDepartment());
            existing.setAge(s.getAge());
            return ResponseEntity.ok(repo.save(existing));
        }).orElse(ResponseEntity.notFound().build());
    }

    // DELETE
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable String id) {
        if (!repo.existsById(id)) {
            return ResponseEntity.notFound().build();
        }
        repo.deleteById(id);
        return ResponseEntity.noContent().build();
    }

    // SEARCH / FILTER  ->  GET /api/students/department/CSE
    @GetMapping("/department/{dept}")
    public List<Student> byDepartment(@PathVariable String dept) {
        return repo.findByDepartment(dept);
    }

    // QUERY PARAM  ->  GET /api/students/search?name=ra
    @GetMapping("/search")
    public List<Student> search(@RequestParam String name) {
        return repo.findByNameContainingIgnoreCase(name);
    }
}
```

Optional service layer

If the question asks for layered architecture, move the logic out of the controller.

```
@Service
@RequiredArgsConstructor
public class StudentService {
    private final StudentRepository repo;

    public List<Student> getAll()          { return repo.findAll(); }
    public Student save(Student s)         { return repo.save(s); }
    public Student get(String id) {
        return repo.findById(id)
                   .orElseThrow(() -> new RuntimeException("Student not
found: " + id));
    }
    public void delete(String id)          { repo.deleteById(id); }
}
```

The controller then injects `StudentService` instead of the repository.

Main class and CORS

```
@SpringBootApplication
public class StudentApplication {
    public static void main(String[] args) {
        SpringApplication.run(StudentApplication.class, args);
    }
}
```

CORS, explained. The Vue app runs on port 5173 and the API on port 8080. A browser treats different ports as different "origins" and blocks the call unless the server says it is allowed. `@CrossOrigin(origins = "http://localhost:5173")` on the controller class is that permission. Without it, the browser console shows a red *CORS policy* error even though the API itself works. Put it on every controller class (no global config file is used).

Annotation cheat list

Annotation	Meaning
@SpringBootApplication	config + auto-config + component scan, placed on the main class
@RestController	@Controller + @ResponseBody, returns JSON
@RequestMapping	base URL for the class

Annotation	Meaning
@GetMapping / @PostMapping / @PutMapping / @DeleteMapping	HTTP method shortcuts
@PathVariable	reads `{id}` from the URL
@RequestParam	reads `?name=x`
@RequestBody	converts incoming JSON to a Java object
@Document / @Id	marks a Mongo collection class / the primary key
@Service / @Repository / @Component	stereotype beans picked up by scanning
@Autowired	field or constructor injection (Lombok's @RequiredArgsConstructor replaces it)
@CrossOrigin	allows the Vue origin to call the API

Test the API before building Vue

```
curl http://localhost:8080/api/students

curl -X POST http://localhost:8080/api/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Hemnath","email":"h@x.com","department":"CSE","age":22}'

curl -X DELETE http://localhost:8080/api/students/<id>
```

Backend code explained line by line

How Spring Boot starts. `SpringApplication.run(...)` in the main class starts an embedded Tomcat on port 8080, scans every package under `com.exam.student` for annotated classes (`@RestController` , `@Service` , …), creates one object of each (a *bean*), connects them (injection), and connects to MongoDB using the URI in `application.properties` . You never start Tomcat yourself.

Model (`Student.java`)

Line	What it does
`@Data`	Lombok generates getters, setters, `toString` , `equals` , `hashCode`

Line	What it does
`@NoArgsConstructor`	empty constructor; Jackson (JSON library) needs it to build a Student from JSON
`@AllArgsConstructor`	constructor with every field, handy for tests
`@Document(collection = "students")`	this class is stored in the Mongo collection named `students`
`@Id private String id;`	marks the primary key. Leave it `null` when creating: Mongo generates it. Its Mongo name is `_id`
other fields	each becomes a field in the Mongo document with the same name

Repository (`StudentRepository.java`)

- `extends MongoRepository<Student, String>` : first type is the class stored, second is the type of the id.
- It gives you ready-made `save` , `findAll` , `findById` , `deleteById` and more with no code.
- `findByDepartment(String department)` : Spring reads the method name (`find` + `By` + `Department`) and builds the query `{ department: ? }` itself. The field name in the method must match the field name in the class, with a capital first letter.

Controller (`StudentController.java`)

Line	What it does
`@RestController`	this class handles web requests and returns data (JSON), not web pages
`@RequestMapping("/api/students")`	every URL in this class starts with `/api/students`
`@CrossOrigin(origins = "http://localhost:5173")`	lets the Vue app on port 5173 call this API
`@RequiredArgsConstructor` + `private final StudentRepository repo;`	Spring injects the repository object into `repo`
`@PostMapping` on `create`	runs for `POST /api/students`
`@RequestBody Student student`	reads the JSON body of the request and converts it to a `Student` object

Line	What it does
`repo.save(student)`	no id means insert; an existing id means update. Returns the saved object (now with its id)
`ResponseEntity.status(HttpStatus.CREATED).body(saved)`	sends status 201 and the saved student as JSON
`@GetMapping` on `getAll`	runs for `GET /api/students`; returning a `List<Student>` becomes a JSON array
`@GetMapping("/{id}")` and `@PathVariable String id`	`{id}` in the URL is copied into the `id` variable
`repo.findById(id)`	returns `Optional<Student>` (may be empty)
`.map(ResponseEntity::ok)`	if found, wrap in a 200 response
`.orElse(ResponseEntity.notFound().build())`	if not found, send 404
`@PutMapping("/{id}")`	update: find the old record, copy new values into it with the setters, save it
`@DeleteMapping("/{id}")`	`existsById` checks it is there, `deleteById` removes it, `noContent()` sends 204
`@RequestParam String name`	reads `?name=ra` from the URL

Why update copies fields instead of saving the incoming object. The incoming JSON may not contain the id. Loading the existing record, changing its fields and saving it guarantees the same Mongo `_id` is reused, so MongoDB updates instead of inserting a duplicate.

What the browser receives for `GET /api/students`:

```
[
  { "id": "66f1...f6", "name": "Hemnath", "email": "h@x.com", "department": "CSE", "age": 22 }
]
```

Note the field is `id` in JSON (Java name), even though Mongo stores `_id`. That is why Vue uses `s.id`.

Common backend questions in a lab

- *Add a new field (say `phone`)*: add `private String phone;` to the model, add it to the form in Vue, nothing else changes.
- *Add a search endpoint*: add a method to the repository (`findByNameContainingIgnoreCase`) and a `@GetMapping("/search")` that calls it.
- *Return a custom message*: return `ResponseEntity.ok("Deleted")` and change the method return type to `ResponseEntity<String>`.
- *Validate input*: check `if (s.getName() == null || s.getName().isBlank())` and return `ResponseEntity.badRequest().build()`.

MongoDB essentials

MongoDB stores JSON-like **documents** in **collections** inside a **database**. There are no tables, rows or fixed schema.

SQL term	MongoDB term
Database	Database
Table	Collection
Row	Document
Column	Field
Primary key	`_id` (auto ObjectId)

A stored student looks like this (`_class` is added by Spring Data):

```
{
  "_id": { "$oid": "66f1c2a9b4e3a1a2c3d4e5f6" },
  "name": "Hemnath",
  "email": "h@x.com",
  "department": "CSE",
  "age": 22,
  "_class": "com.exam.student.model.Student"
}
```

Mongo shell commands (mongosh)

```
show dbs
use studentdb
show collections

db.students.insertOne({ name: "Asha", email: "a@x.com", department: "IT",
age: 21 })
db.students.insertMany([ { name: "B", age: 20 }, { name: "C", age: 23 } ])

db.students.find()
db.students.find({ department: "CSE" })
db.students.find({ age: { $gt: 21 } })
db.students.findOne({ email: "a@x.com" })

db.students.updateOne({ name: "Asha" }, { $set: { age: 22 } })
db.students.deleteOne({ name: "Asha" })
db.students.deleteMany({ department: "IT" })

db.students.countDocuments()
db.students.drop()
```

Query operators to remember: `$gt $gte $lt $lte $ne $in $and $or $set $inc`.

Spring Data derived query keywords

Method name	Meaning
findByName(String n)	name equals n
findByAgeGreaterThan(int a)	age > a
findByAgeBetween(int a, int b)	a < age < b
findByNameContaining(String s)	name contains s
findByNameContainingIgnoreCase	case-insensitive contains
findByDepartmentAndAge(String d, int a)	both conditions
findByDepartmentOrAge(String d, int a)	either condition
findByDepartmentOrderByNameAsc	sorted result
existsByEmail / countByDepartment / deleteByEmail	exists / count / delete derived queries

Custom JSON query:

```
@Query("{ 'department': ?0, 'age': { $gt: ?1 } }")
List<Student> findSeniors(String dept, int age);
```

Using MongoTemplate (if asked)

```
@Autowired private MongoTemplate mongoTemplate;

Query q = new Query(Criteria.where("department").is("CSE"));
List<Student> list = mongoTemplate.find(q, Student.class);
```

Vue 3 fundamentals

Project setup

```
npm create vue@latest frontend      # choose: Router = Yes, Pinia = Yes
cd frontend
npm install
npm install axios                   # HTTP client
# if Router/Pinia were not chosen at creation:
npm install vue-router@4 pinia
npm run dev                         # http://localhost:5173
```

Entry files

`index.html` holds `<div id="app"></div>` and loads `/src/main.js`.

`main.js` creates the app and plugs in Pinia and the router:

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import router from './router'
import './assets/style.css'

const app = createApp(App)
app.use(createPinia())     // state management
app.use(router)            // routing
app.mount('#app')
```

`App.vue` is the root component and holds the navbar plus `<RouterView />`:

```
<template>
  <header class="navbar">
    <h1>Student Manager</h1>
    <nav>
      <RouterLink to="/">Students</RouterLink>
      <RouterLink to="/add">Add Student</RouterLink>
    </nav>
  </header>
  <main class="container">
    <RouterView />
  </main>
</template>
```

Single File Component (SFC) anatomy

```
<script setup>
// logic: imports, state, functions
import { ref } from 'vue'
const count = ref(0)
function inc() { count.value++ }
</script>

<template>
  <!-- markup -->
  <button @click="inc">Clicked {{ count }}</button>
</template>

<style scoped>
/* CSS only for this component */
button { color: white; background: #2563eb; }
</style>
```

Reactivity

API	Use
`ref(value)`	reactive primitive or object; read and write with `.value` in script, no `.value` in template
`reactive({})`	reactive object, no `.value` needed
`computed(() => ...)`	derived value that caches
`watch(source, (n, o) => ...)`	run code when a value changes

API	Use
`onMounted(() => ...)`	run after component is rendered (use for first data load)
`storeToRefs(store)`	destructure Pinia state without losing reactivity

Template syntax and directives

Directive	Example	Meaning
`{{ }}`	`{{ student.name }}`	text interpolation
`v-bind` / `:`	`:class="{ active: isOn }"`, `:src="url"`	bind attribute to data
`v-on` / `@`	`@click="remove(s.id)"`, `@submit.prevent="save"`	event handler
`v-model`	`<input v-model="form.name">`	two-way binding for inputs
`v-if / v-else-if / v-else`	`<p v-if="list.length === 0">No data</p>`	conditional render
`v-show`	`v-show="visible"`	toggles CSS display
`v-for`	`<tr v-for="s in students" :key="s.id">`	loop, `:key` is mandatory
`v-model.number` / `.trim`	`v-model.number="form.age"`	modifiers

Props and emits (parent and child components)

```
<!-- StudentCard.vue (child) -->
<script setup>
const props = defineProps({ student: Object })
const emit = defineEmits(['delete'])
</script>
<template>
  <div class="card">
    <h3>{{ student.name }}</h3>
    <button @click="emit('delete', student.id)">Delete</button>
  </div>
</template>

<!-- parent -->
<StudentCard v-for="s in students" :key="s.id"
             :student="s" @delete="removeStudent" />
```

Data flows **down** by props and **up** by emitted events.

Vue Router

Vue Router maps a URL to a component and swaps it inside `<RouterView />` without reloading the page.

router/index.js

```
import { createRouter, createWebHistory } from 'vue-router'
import StudentList from '../views/StudentList.vue'
import StudentForm from '../views/StudentForm.vue'

const routes = [
  { path: '/',          name: 'students', component: StudentList },
  { path: '/add',       name: 'add',      component: StudentForm },
  { path: '/edit/:id',  name: 'edit',     component: StudentForm, props: true
},
  // lazy loaded route
  { path: '/about',     name: 'about',    component: () =>
import('../views/About.vue') },
  // catch-all 404
  { path: '/:pathMatch(.*)*', name: 'notfound', component: () =>
import('../views/NotFound.vue') }
]

const router = createRouter({
```

```
  history: createWebHistory(),   // clean URLs; createWebHashHistory() gives
/#/path
  routes
})

export default router
```

Route object options

Option	Purpose
`path`	URL pattern; `:id` is a dynamic segment
`name`	lets you navigate by name
`component`	view to render
`props: true`	passes route params to the component as props
`redirect`	`{ path: '/home', redirect: '/' }`
`meta`	custom data, e.g. `{ requiresAuth: true }`
`children`	nested routes rendered in a nested `<RouterView />`

Navigating

```
<!-- template: declarative -->
<RouterLink to="/add">Add</RouterLink>
<RouterLink :to="{ name: 'edit', params: { id: s.id } }">Edit</RouterLink>

<script setup>
import { useRouter, useRoute } from 'vue-router'
const router = useRouter()    // navigate
const route  = useRoute()     // read current route

router.push('/')                                  // go to path
router.push({ name: 'edit', params: { id: 5 } })  // go by name
router.replace('/')                               // no history entry
router.back()                                     // browser back

const id = route.params.id        // /edit/:id
const q  = route.query.search     // /?search=abc
</script>
```

`RouterLink` adds the class `router-link-active` (and `router-link-exact-active`) to the current link, so style it for the navbar highlight.

Navigation guard (if asked)

```
router.beforeEach((to, from) => {
  const loggedIn = localStorage.getItem('token')
  if (to.meta.requiresAuth && !loggedIn) {
    return '/login'          // redirect
  }
  // returning nothing allows navigation
})
```

In-component guards: `onBeforeRouteLeave`, `onBeforeRouteUpdate` from `vue-router`.

Pinia store and axios

Pinia holds shared state outside components. Three parts: **state** (data), **getters** (computed values), **actions** (functions, sync or async). Here the actions call the Spring Boot API with axios, so components never touch axios directly.

stores/studentStore.js (options style)

```
import { defineStore } from 'pinia'
import axios from 'axios'

const API = 'http://localhost:8080/api/students'

export const useStudentStore = defineStore('student', {
  state: () => ({
    students: [],
    student: null,        // the one being edited
    loading: false,
    error: null
  }),

  getters: {
    total: (state) => state.students.length,
    byDepartment: (state) => (dept) =>
      state.students.filter((s) => s.department === dept)
  },

  actions: {
    async fetchAll() {
```

```
      this.loading = true
      this.error = null
      try {
        const res = await axios.get(API)
        this.students = res.data
      } catch (e) {
        this.error = e.message
      } finally {
        this.loading = false
      }
    },

    async fetchOne(id) {
      const res = await axios.get(`${API}/${id}`)
      this.student = res.data
    },

    async addStudent(s) {
      await axios.post(API, s)
      await this.fetchAll()
    },

    async updateStudent(id, s) {
      await axios.put(`${API}/${id}`, s)
      await this.fetchAll()
    },

    async deleteStudent(id) {
      await axios.delete(`${API}/${id}`)
      this.students = this.students.filter((s) => s.id !== id)
    }
  }
})
```

Same store in setup style (also accepted)

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export const useStudentStore = defineStore('student', () => {
  const students = ref([])                          // state
  const total = computed(() => students.value.length)  // getter
  async function fetchAll() {                       // action
    students.value = (await axios.get(API)).data
  }
  return { students, total, fetchAll }
})
```

Using the store in a component

```
import { storeToRefs } from 'pinia'
import { useStudentStore } from '../stores/studentStore'

const store = useStudentStore()
const { students, loading, total } = storeToRefs(store)   // state + getters
stay reactive
store.fetchAll()                                          // actions are
called directly
```

Do not destructure state directly (`const { students } = store`): it loses reactivity. Actions can be destructured safely.

axios quick reference

Call	Meaning
`axios.get(url)`	read
`axios.post(url, body)`	create
`axios.put(url, body)`	update
`axios.delete(url)`	delete
`res.data`	JSON body returned by Spring
`axios.create({ baseURL })`	reusable instance with a base URL

```
// optional src/api.js
import axios from 'axios'
export default axios.create({ baseURL: 'http://localhost:8080/api' })
```

Components and styling

Two views cover the whole CRUD: a list (read + delete) and a form that serves both add and edit.

views/StudentList.vue

```
<script setup>
import { onMounted } from 'vue'
import { storeToRefs } from 'pinia'
import { useStudentStore } from '../stores/studentStore'

const store = useStudentStore()
const { students, loading, error } = storeToRefs(store)

onMounted(() => store.fetchAll())          // load data when page opens

async function remove(id) {
  if (confirm('Delete this student?')) {
    await store.deleteStudent(id)
  }
}
</script>

<template>
  <section>
    <h2>All Students ({{ students.length }})</h2>

    <p v-if="loading">Loading...</p>
    <p v-else-if="error" class="error">{{ error }}</p>
    <p v-else-if="students.length === 0">No students found.</p>

    <table v-else class="table">
      <thead>
        <tr>
          <th>Name</th><th>Email</th><th>Department</th><th>Age</th>
<th>Actions</th>
        </tr>
      </thead>
      <tbody>
```

```
        <tr v-for="s in students" :key="s.id">
          <td>{{ s.name }}</td>
          <td>{{ s.email }}</td>
          <td>{{ s.department }}</td>
          <td>{{ s.age }}</td>
          <td>
            <RouterLink :to="{ name: 'edit', params: { id: s.id } }"
class="btn edit">Edit</RouterLink>
            <button class="btn delete" @click="remove(s.id)">Delete</button>
          </td>
        </tr>
      </tbody>
    </table>
  </section>
</template>
```

views/StudentForm.vue (add and edit in one)

```
<script setup>
import { reactive, computed, onMounted } from 'vue'
import { useRouter } from 'vue-router'
import { useStudentStore } from '../stores/studentStore'

const props = defineProps({ id: String })      // present only on /edit/:id
const store = useStudentStore()
const router = useRouter()

const isEdit = computed(() => !!props.id)
const form = reactive({ name: '', email: '', department: '', age: null })

onMounted(async () => {
  if (isEdit.value) {
    await store.fetchOne(props.id)
    Object.assign(form, store.student)           // pre-fill the form
  }
})

async function submit() {
  if (isEdit.value) {
    await store.updateStudent(props.id, form)
  } else {
    await store.addStudent(form)
  }
  router.push('/')                               // back to the list
}
</script>
```

```
<template>
  <section>
    <h2>{{ isEdit ? 'Edit Student' : 'Add Student' }}</h2>

    <form class="form" @submit.prevent="submit">
      <label>Name
        <input v-model.trim="form.name" required />
      </label>
      <label>Email
        <input v-model.trim="form.email" type="email" required />
      </label>
      <label>Department
        <select v-model="form.department" required>
          <option disabled value="">Select</option>
          <option>CSE</option>
          <option>IT</option>
          <option>ECE</option>
        </select>
      </label>
      <label>Age
        <input v-model.number="form.age" type="number" min="1" required />
      </label>

      <button class="btn save" type="submit">{{ isEdit ? 'Update' : 'Save' }}
</button>
      <RouterLink to="/" class="btn cancel">Cancel</RouterLink>
    </form>
  </section>
</template>
```

Key points: `@submit.prevent` stops the page reload, `v-model.number` keeps age numeric (otherwise it is sent as a string), and `Object.assign(form, store.student)` fills the edit form.

assets/style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: Arial, sans-serif; background: #f3f4f6; color: #111827; }

.navbar {
  display: flex; justify-content: space-between; align-items: center;
  background: #1e3a8a; color: #fff; padding: 14px 24px;
}
.navbar a { color: #fff; margin-left: 16px; text-decoration: none; }
.navbar a.router-link-exact-active { border-bottom: 2px solid #fbbf24; }
```

```
.container { max-width: 900px; margin: 24px auto; padding: 0 16px; }

.table { width: 100%; border-collapse: collapse; background: #fff; }
.table th, .table td { padding: 10px; border-bottom: 1px solid #e5e7eb; text-
align: left; }
.table th { background: #e5e7eb; }
.table tr:hover { background: #f9fafb; }

.form { display: grid; gap: 12px; max-width: 420px; background: #fff;
padding: 20px; border-radius: 8px; }
.form label { display: grid; gap: 4px; font-weight: bold; }
.form input, .form select { padding: 8px; border: 1px solid #9ca3af; border-
radius: 4px; }

.btn { padding: 6px 12px; border: none; border-radius: 4px; cursor: pointer;
color: #fff; text-decoration: none; display: inline-block; }
.save   { background: #16a34a; }
.edit   { background: #2563eb; margin-right: 6px; }
.delete { background: #dc2626; }
.cancel { background: #6b7280; }
.error  { color: #dc2626; }
```

Style ideas Vue questions ask for: dynamic class `:class="{ active: isOn }"`, inline style `:style="{ color: c }"`, `<style scoped>` for component-only CSS, `.router-link-active` for the current nav link.

Basic HTML/CSS reminders

Element or property	Use
`<form>`, `<input>`, `<select>`, `<textarea>`, `<button>`	data entry
`<table> <thead> <tbody> <tr> <th> <td>`	tabular list
`display: flex` / `grid`	layout
`padding`, `margin`, `border`, `border-radius`	box model
`:hover`, `:focus`	interaction states
`@media (max-width: 600px) { ... }`	responsive design

Frontend code explained line by line

How the app boots. The browser opens `index.html`, which has an empty `<div id="app">`. `main.js` creates the Vue app, registers Pinia and the router, and mounts it into that div. `App.vue` is the first component drawn; the router then puts the matching view inside `<RouterView />`.

`main.js`

Line	Meaning
`createApp(App)`	make a Vue application whose root component is `App.vue`
`app.use(createPinia())`	install the state store system so `useStudentStore()` works anywhere
`app.use(router)`	install the router so `<RouterLink>`, `<RouterView>`, `useRouter()` work
`app.mount('#app')`	draw the app inside the element with id `app`

`router/index.js`

- `routes` is an array; each object says "when the URL is `path`, show `component`".
- `/edit/:id` has a *dynamic segment*: `:id` matches any value (`/edit/66f1...`) and the value is available as `route.params.id`.
- `props: true` hands that `id` to the component as a prop, so `defineProps({ id: String })` receives it.
- `createWebHistory()` gives normal URLs without `#`.
- Importing both views at the top and using them in `routes` is how the router knows which file to show.

`stores/studentStore.js`

Part	Meaning
`defineStore('student', {...})`	creates a store; the first argument is a unique name
`export const useStudentStore`	the function every component calls to get the store
`state: () => ({ ... })`	the data. Function returning an object so each app gets a fresh copy

Part	Meaning
`getters`	computed values built from state, like `total`
`actions`	functions that change state or call the API; use `this.` to reach state
`async` / `await`	wait for the server before moving to the next line
`try / catch / finally`	`try` runs the call, `catch` stores any error, `finally` always turns the loading flag off
`res.data`	axios puts the JSON body in `data`
template string `` `${API}/${id}` ``	builds `http://localhost:8080/api/students/66f1...`
`this.students.filter(s => s.id !== id)`	after delete, remove that student from the local list so the page updates instantly

`StudentList.vue`

Line	Meaning
`const store = useStudentStore()`	get the shared store
`storeToRefs(store)`	pull out `students`, `loading`, `error` as reactive refs
`onMounted(() => store.fetchAll())`	when the page appears, ask the backend for data
`v-if / v-else-if / v-else`	show only one of: loading text, error text, empty text, table
`v-for="s in students" :key="s.id"`	repeat the `<tr>` once per student; `key` helps Vue track rows
`{{ s.name }}`	print the value into the page
`<RouterLink :to="{ name: 'edit', params: { id: s.id } }">`	link to the edit page for that student (the colon before `to` means the value is an expression, not text)
`@click="remove(s.id)"`	call `remove` with this row's id when clicked
`confirm(...)`	browser popup that returns true if user presses OK

`StudentForm.vue`

Line	Meaning
`defineProps({ id: String })`	receive `id` from the route; it exists only in edit mode
`computed(() => !!props.id)`	`!!` turns the value into true or false; true means edit mode
`reactive({ name: '', ... })`	the form's data object; inputs write into it
`v-model="form.name"`	two-way link: typing updates `form.name`, changing `form.name` updates the input
`v-model.trim` / `.number`	remove spaces / convert text to a number
`@submit.prevent="submit"`	run `submit` and stop the browser's default page reload
`Object.assign(form, store.student)`	copy all fields of the loaded student into the form so inputs show old values
`isEdit ? 'Update' : 'Save'`	ternary: pick the text based on mode
`router.push('/')`	go back to the list after saving

Why list reloads after add: `addStudent` calls `fetchAll()` at the end, and `StudentList` also calls `fetchAll()` on mount, so the new row is always present.

Typical changes asked in the test

- *Add a field*: add it to `form`, add an `<input v-model="form.phone">`, add a `<th>` and `<td>` in the list.
- *Add a search box*: `const q = ref('')`, `<input v-model="q">`, and `const filtered = computed(() => students.value.filter(s => s.name.toLowerCase().includes(q.value.toLowerCase())))`, then loop over `filtered`.
- *Add a new page*: create `views/About.vue`, add a route object, add a `<RouterLink>`.
- *Show a message after saving*: `const msg = ref('')`, set `msg.value = 'Saved'`, show `<p v-if="msg">{{ msg }}</p>`.

Running it, full flow, common errors

Run commands

```
# 1. MongoDB (start the service or:)
mongod                        # or: net start MongoDB (Windows) / brew
services start mongodb-community

# 2. Backend
cd backend
mvn spring-boot:run           # http://localhost:8080

# 3. Frontend
cd frontend
npm install
npm run dev                   # http://localhost:5173
```

What happens when you click "Save"

1. `v-model` has already copied the input values into `form`.
2. `@submit.prevent` calls `submit()`.
3. `submit()` calls `store.addStudent(form)` (Pinia action).
4. The action runs `axios.post('http://localhost:8080/api/students', form)`.
5. Spring's `@PostMapping` receives the JSON; `@RequestBody` turns it into a `Student`.
6. `repo.save()` inserts into MongoDB and returns the saved object with an `id`.
7. The action calls `fetchAll()`, which fills `store.students`.
8. `router.push('/')` opens the list; `v-for` renders the new row.

Exam checklist

- [ ] MongoDB running on 27017
- [ ] `spring.data.mongodb.uri` has the database name
- [ ] Model has `@Document`, `@Id`, `@Data`, `@NoArgsConstructor`
- [ ] Repository extends `MongoRepository<Student, String>`
- [ ] Controller has `@RestController`, `@RequestMapping`, `@CrossOrigin`
- [ ] API tested in browser or Postman first
- [ ] `main.js` has `app.use(createPinia())` and `app.use(router)`
- [ ] `App.vue` has `<RouterView />`
- [ ] Router has `/`, `/add`, `/edit/:id`
- [ ] Store actions use `await` and the right URL

- [ ] `v-for` has `:key`, number input uses `v-model.number`

Common errors and fixes

Symptom	Cause	Fix
CORS error in browser console	origin not allowed	add `@CrossOrigin(origins = "http://localhost:5173")`
Getter/setter not found, red code	Lombok not processed	install Lombok plugin, enable annotation processing, rebuild
`MongoTimeoutException`	Mongo is not running	start `mongod`, check URI and port
404 on API call	wrong URL or missing `/api`	match axios URL to `@RequestMapping`
415 Unsupported Media Type	JSON not sent	axios sends JSON by default; check `@RequestBody`
Edit form is empty	store not loaded	call `fetchOne(id)` in `onMounted`, then `Object.assign`
Blank page, no error	`<RouterView />` missing	add it in `App.vue`
`getActivePinia was called with no active Pinia`	Pinia not installed	`app.use(createPinia())` before `mount`
Update creates a duplicate	id not sent in the URL	use PUT with `/{id}` and `findById` + `save(existing)`
Port 8080 in use	another app running	change `server.port` or stop the other process

Part B: Servlets with MySQL (JDBC)

A servlet is a Java class that runs inside a servlet container (Tomcat), receives an HTTP request and writes an HTTP response. The container creates it, calls it, and destroys it.

Package names depend on Tomcat: Tomcat 9 and older use `javax.servlet.*`; Tomcat 10+ uses `jakarta.servlet.*`. Use whichever your lab's Tomcat needs; the code is otherwise identical. Examples below use `jakarta`.

Architecture and life cycle

```
Browser --HTTP request--> Tomcat (web container)
                            |  maps URL to servlet (web.xml or @WebServlet)
                            v
                    Servlet: init() once -> service() per request
                             -> doGet() / doPost() -> destroy() once
                             |
                             v
                    JDBC -> MySQL ; response HTML back to browser
```

Method	Called	Purpose
`init(ServletConfig)`	once, when first loaded	open resources, read init params
`service(req, res)`	every request	dispatches to doGet, doPost, etc.
`doGet(req, res)`	GET request	read data, show pages
`doPost(req, res)`	POST request	form submit, insert/update
`destroy()`	once, at unload	release resources

One servlet instance serves many requests on many threads, so avoid shared mutable instance fields.

Setup (Maven web project)

Folder layout:

```
servletdemo/
  pom.xml
  src/main/java/com/exam/
    DBConnection.java, Student.java, StudentDAO.java
    HelloServlet.java, AddStudentServlet.java, ListStudentServlet.java ...
  src/main/webapp/
    index.html, addStudent.html
    WEB-INF/web.xml
```

```
<packaging>war</packaging>
<dependencies>
    <dependency>
        <groupId>jakarta.servlet</groupId>
        <artifactId>jakarta.servlet-api</artifactId>
        <version>6.0.0</version>
        <scope>provided</scope>
    </dependency>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>8.4.0</version>
    </dependency>
</dependencies>
```

In Eclipse (Dynamic Web Project) instead: copy `mysql-connector-j.jar` into `WEB-INF/lib`, and add Tomcat as the server runtime.

Hello Servlet

```
package com.exam;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.io.PrintWriter;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");   // /hello?name=Hemnath
        if (name == null) name = "World";

        out.println("<html><body>");
```

```
        out.println("<h1>Hello, " + name + "!</h1>");
        out.println("</body></html>");
    }
}
```

Run on Tomcat, open `http://localhost:8080/servletdemo/hello?name=Hemnath`.

Same mapping with web.xml (use this if the question says not to use annotations):

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee" version="6.0">
    <servlet>
        <servlet-name>hello</servlet-name>
        <servlet-class>com.exam.HelloServlet</servlet-class>
        <init-param>
            <param-name>college</param-name>
            <param-value>SSNCE</param-value>
        </init-param>
    </servlet>
    <servlet-mapping>
        <servlet-name>hello</servlet-name>
        <url-pattern>/hello</url-pattern>
    </servlet-mapping>
    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>
</web-app>
```

Reading the init param: `getServletConfig().getInitParameter("college")`. Context-wide params live in `<context-param>` and are read with `getServletContext().getInitParameter(...)`.

Request and response methods

Object	Method	Use
request	`getParameter("x")`	single form value (String)
request	`getParameterValues("x")`	checkboxes, multi-select (String[])
request	`getParameterNames()`, `getParameterMap()`	all parameters
request	`getMethod()`, `getRequestURI()`, `getHeader("User-Agent")`	request info
request	`setAttribute / getAttribute`	pass objects to a forwarded page

Object	Method	Use
request	`getSession()`, `getCookies()`	session and cookie access
request	`getRequestDispatcher("x").forward(req, res)`	server-side forward
response	`setContentType("text/html")`	MIME type
response	`getWriter()`	text output (`getOutputStream()` for binary)
response	`sendRedirect("url")`	browser redirect (new request)
response	`addCookie(c)`, `setHeader(k, v)`, `setStatus(n)`	extras

`forward` stays on the server and keeps the URL; `sendRedirect` makes the browser issue a new request and the URL changes. `include` adds another resource's output to the current response.

HTML form that posts to a servlet

```
<form action="addStudent" method="post">
    Name: <input type="text" name="name" required><br>
    Email: <input type="email" name="email" required><br>
    Department: <input type="text" name="department"><br>
    Age: <input type="number" name="age"><br>
    <input type="submit" value="Save">
</form>
```

The `name` attribute of each input is the key read by `request.getParameter("name")`.

MySQL table

```
CREATE DATABASE studentdb;
USE studentdb;

CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    department VARCHAR(50),
    age INT
);

INSERT INTO students (name, email, department, age) VALUES ('Asha',
'a@x.com', 'CSE', 21);
SELECT * FROM students;
UPDATE students SET age = 22 WHERE id = 1;
DELETE FROM students WHERE id = 1;
```

JDBC steps

1. Load the driver: `Class.forName("com.mysql.cj.jdbc.Driver")` (optional in JDBC 4+, but safe).
2. Open a `Connection` with `DriverManager.getConnection(url, user, password)`.
3. Create a `PreparedStatement` with `?` placeholders.
4. Set values with `ps.setString(1, ...)`, `ps.setInt(2, ...)`.
5. Run: `executeQuery()` for SELECT (returns `ResultSet`), `executeUpdate()` for INSERT/UPDATE/DELETE (returns affected row count).
6. Read results with `rs.next()` and `rs.getInt("id")` / `rs.getString("name")`.
7. Close everything (use try-with-resources).

DBConnection.java

```
package com.exam;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    private static final String URL =
        "jdbc:mysql://localhost:3306/studentdb?
useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true";
    private static final String USER = "root";
    private static final String PASS = "root";      // use your own password

    public static Connection getConnection() throws SQLException,
ClassNotFoundException {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

Student.java (plain POJO)

```
package com.exam;

public class Student {
    private int id;
    private String name, email, department;
    private int age;

    public Student() {}
    public Student(int id, String name, String email, String department, int
age) {
        this.id = id; this.name = name; this.email = email;
        this.department = department; this.age = age;
    }
    public int getId() { return id; }          public void setId(int id) {
this.id = id; }
    public String getName() { return name; }   public void setName(String n)
{ this.name = n; }
    public String getEmail() { return email; } public void setEmail(String e)
{ this.email = e; }
    public String getDepartment() { return department; }
    public void setDepartment(String d) { this.department = d; }
    public int getAge() { return age; }        public void setAge(int a) {
this.age = a; }
}
```

StudentDAO.java (all CRUD)

```
package com.exam;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class StudentDAO {

    public int add(Student s) throws Exception {
        String sql = "INSERT INTO students(name, email, department, age)
VALUES (?,?,?,?)";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, s.getName());
            ps.setString(2, s.getEmail());
            ps.setString(3, s.getDepartment());
            ps.setInt(4, s.getAge());
```

```
            return ps.executeUpdate();
        }
    }

    public List<Student> getAll() throws Exception {
        List<Student> list = new ArrayList<>();
        try (Connection con = DBConnection.getConnection();
             Statement st = con.createStatement();
             ResultSet rs = st.executeQuery("SELECT * FROM students")) {
            while (rs.next()) {
                list.add(new Student(rs.getInt("id"), rs.getString("name"),
                        rs.getString("email"), rs.getString("department"),
rs.getInt("age")));
            }
        }
        return list;
    }

    public Student getById(int id) throws Exception {
        String sql = "SELECT * FROM students WHERE id = ?";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return new Student(rs.getInt("id"), rs.getString("name"),
                            rs.getString("email"),
rs.getString("department"), rs.getInt("age"));
                }
            }
        }
        return null;
    }

    public int update(Student s) throws Exception {
        String sql = "UPDATE students SET name=?, email=?, department=?,
age=? WHERE id=?";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, s.getName());
            ps.setString(2, s.getEmail());
            ps.setString(3, s.getDepartment());
            ps.setInt(4, s.getAge());
            ps.setInt(5, s.getId());
            return ps.executeUpdate();
        }
```

```
    }

    public int delete(int id) throws Exception {
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement("DELETE FROM
students WHERE id=?")) {
            ps.setInt(1, id);
            return ps.executeUpdate();
        }
    }
}
```

`PreparedStatement` with `?` is preferred over `Statement` because it blocks SQL injection and is precompiled.

CRUD servlets

Insert

```
@WebServlet("/addStudent")
public class AddStudentServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        Student s = new Student();
        s.setName(req.getParameter("name"));
        s.setEmail(req.getParameter("email"));
        s.setDepartment(req.getParameter("department"));
        s.setAge(Integer.parseInt(req.getParameter("age")));
        try {
            new StudentDAO().add(s);
            res.sendRedirect("listStudents");        // show updated list
        } catch (Exception e) {
            res.setContentType("text/html");
            res.getWriter().println("<p>Error: " + e.getMessage() + "</p>");
        }
    }
}
```

List

```
@WebServlet("/listStudents")
public class ListStudentServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
```

```
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        try {
            List<Student> list = new StudentDAO().getAll();
            out.println("<h2>Students</h2>");
            out.println("<table border='1' cellpadding='6'>");
            out.println("<tr><th>ID</th><th>Name</th><th>Email</th>
<th>Dept</th><th>Age</th><th>Action</th></tr>");
            for (Student s : list) {
                out.println("<tr><td>" + s.getId() + "</td><td>" +
s.getName() + "</td><td>"
                        + s.getEmail() + "</td><td>" + s.getDepartment() + "
</td><td>" + s.getAge()
                        + "</td><td><a href='editStudent?id=" + s.getId() +
"'>Edit</a> "
                        + "<a href='deleteStudent?id=" + s.getId() +
"'>Delete</a></td></tr>");
            }
            out.println("</table><a href='addStudent.html'>Add new</a>");
        } catch (Exception e) {
            out.println("Error: " + e.getMessage());
        }
    }
}
```

Edit (GET shows prefilled form, POST saves)

```
@WebServlet("/editStudent")
public class EditStudentServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        try {
            Student s = new
StudentDAO().getById(Integer.parseInt(req.getParameter("id")));
            out.println("<form action='editStudent' method='post'>");
            out.println("<input type='hidden' name='id' value='" + s.getId()
+ "'>");
            out.println("Name: <input name='name' value='" + s.getName() +
"'><br>");
            out.println("Email: <input name='email' value='" + s.getEmail() +
"'><br>");
            out.println("Dept: <input name='department' value='" +
s.getDepartment() + "'><br>");
```

```
            out.println("Age: <input name='age' value='" + s.getAge() + "'>
<br>");
            out.println("<input type='submit' value='Update'></form>");
        } catch (Exception e) { out.println("Error: " + e.getMessage()); }
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        try {
            Student s = new Student(Integer.parseInt(req.getParameter("id")),
                    req.getParameter("name"), req.getParameter("email"),
                    req.getParameter("department"),
Integer.parseInt(req.getParameter("age")));
            new StudentDAO().update(s);
            res.sendRedirect("listStudents");
        } catch (Exception e) { res.getWriter().println("Error: " +
e.getMessage()); }
    }
}
```

Delete

```
@WebServlet("/deleteStudent")
public class DeleteStudentServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        try {
            new
StudentDAO().delete(Integer.parseInt(req.getParameter("id")));
            res.sendRedirect("listStudents");
        } catch (Exception e) { res.getWriter().println("Error: " +
e.getMessage()); }
    }
}
```

Full flow for "Save"

1. Browser submits `addStudent.html` form with POST to `/addStudent`.
2. Tomcat finds `AddStudentServlet` through `@WebServlet` and calls `service()`, then `doPost()`.
3. The servlet reads parameters and builds a `Student`.
4. `StudentDAO.add()` opens a JDBC connection and runs the INSERT.

5. `sendRedirect("listStudents")` sends the browser to the list servlet.
6. `ListStudentServlet.doGet()` queries MySQL and prints the HTML table.

Session management (Unit III topics)

HTTP is stateless; these four techniques remember a user between requests.

HttpSession

```
// LoginServlet.doPost
HttpSession session = request.getSession();            // creates if absent
session.setAttribute("user", request.getParameter("username"));
session.setMaxInactiveInterval(30 * 60);               // seconds
response.sendRedirect("profile");

// ProfileServlet.doGet
HttpSession session = request.getSession(false);       // do not create
if (session == null || session.getAttribute("user") == null) {
    response.sendRedirect("login.html");
    return;
}
String user = (String) session.getAttribute("user");

// LogoutServlet
request.getSession().invalidate();
```

Cookies

```
// write
Cookie c = new Cookie("username", "hemnath");
c.setMaxAge(60 * 60 * 24);       // 1 day; 0 deletes it; negative = until
browser closes
c.setPath("/");
response.addCookie(c);

// read
Cookie[] cookies = request.getCookies();
if (cookies != null) {
    for (Cookie ck : cookies) {
        if (ck.getName().equals("username")) {
            out.println("Welcome back " + ck.getValue());
        }
    }
}
```

URL rewriting (works when cookies are disabled; the session id is appended to the link)

```
String url = response.encodeURL("profile");     // profile;jsessionid=ABC123
out.println("<a href='" + url + "'>Profile</a>");
```

Hidden form field

```
<input type="hidden" name="userId" value="101">
```

Technique	Stored at	Good for	Weakness
HttpSession	server (id in cookie)	login state, cart	uses server memory
Cookie	browser	preferences, remember me	user can edit or disable
URL rewriting	in the link	cookies disabled	ids visible in URL
Hidden field	in the form	multi-step forms	only works with forms

ServletConfig vs ServletContext

	ServletConfig	ServletContext
Scope	one servlet	whole web app
Params from	`<init-param>`	`<context-param>`
Get it with	`getServletConfig()`	`getServletContext()`

Forwarding to a JSP (optional): the servlet fetches data, stores it in the request, and the JSP draws it.

```
request.setAttribute("students", list);
request.getRequestDispatcher("students.jsp").forward(request, response);
```

```
<%@ page import="java.util.*, com.exam.Student" %>
<% List<Student> list = (List<Student>) request.getAttribute("students");
   for (Student s : list) { %>
    <p><%= s.getName() %> - <%= s.getDepartment() %></p>
<% } %>
```

Common errors

Error	Fix
`ClassNotFoundException: com.mysql.cj.jdbc.Driver`	connector jar missing from `WEB-INF/lib` or pom
`Access denied for user 'root'`	wrong password in `DBConnection`
`Communications link failure`	MySQL service not running or wrong port
404 on servlet	wrong `urlPattern`, wrong context path, or app not redeployed
`javax` vs `jakarta` compile errors	match imports to the Tomcat version (9 = javax, 10+ = jakarta)
`NumberFormatException`	empty or missing `age` parameter; validate before `parseInt`
405 Method Not Allowed	form uses POST but servlet only overrides `doGet`

Servlet code explained line by line

Hello servlet

Line	Meaning
`extends HttpServlet`	makes the class a servlet; Tomcat can now call it
`@WebServlet("/hello")`	URL `/hello` runs this class (same job as `<servlet-mapping>` in web.xml)
`protected void doGet(HttpServletRequest request, HttpServletResponse response)`	runs for GET. `request` holds what the browser sent; `response` is what you send back
`throws ServletException, IOException`	these checked exceptions must be declared
`response.setContentType("text/html")`	tells the browser the reply is an HTML page
`PrintWriter out = response.getWriter()`	a pen for writing the reply body

Line	Meaning
`request.getParameter("name")`	reads `?name=...` from the URL, or a form field; returns `null` if absent
`out.println("<h1>...")`	writes HTML text into the response

Which method runs? Typing a URL in the browser or clicking a link sends GET, so `doGet` runs. A form with `method="post"` sends POST, so `doPost` runs. If the form is POST and you wrote only `doGet`, you get error 405. A common trick is to let both do the same job: `protected void doPost(...) { doGet(req, res); }`.

How the URL maps to the servlet. `http://localhost:8080/servletdemo/hello` splits as host and port, then **context path** `/servletdemo` (the project name), then **servlet path** `/hello` (from `@WebServlet`). If you rename the project or forget the context path you get 404.

DAO pattern. `StudentDAO` holds all database code; servlets hold only request and response code. This separation is why each servlet is short: read parameters, call the DAO, redirect or print.

What each JDBC line in the DAO does

Line	Meaning
`Connection con = DBConnection.getConnection()`	opens a live connection to MySQL
`con.prepareStatement(sql)`	sends the SQL with `?` holes to the database to be prepared
`ps.setString(1, value)`	fills hole number 1 (counting starts at 1, not 0)
`ps.executeUpdate()`	runs INSERT, UPDATE or DELETE and returns how many rows changed
`ps.executeQuery()`	runs SELECT and returns a `ResultSet`
`while (rs.next())`	moves to the next row; false when rows are finished
`rs.getInt("id")`, `rs.getString("name")`	read a column of the current row by its column name
`try (...) { }`	closes connection, statement and result set automatically when the block ends

Why `sendRedirect` after insert. If you simply print a page after a POST, pressing refresh would submit the form again and insert a duplicate. Redirecting makes the browser do a fresh GET to the list page.

Edit flow in detail. The list prints a link `editStudent?id=5`. That is GET, so `doGet` loads student 5 and prints a form with the old values filled in and a hidden field holding the id. Pressing Update sends POST, so `doPost` builds a `Student` including the id and calls `update`, which runs `UPDATE ... WHERE id=?`.

Session in plain words. The first time you call `request.getSession()`, Tomcat creates a session object and sends the browser a cookie named `JSESSIONID`. On later requests the browser sends that cookie back and Tomcat finds the same session, so values saved with `setAttribute` are still there. `invalidate()` destroys it (logout). `getSession(false)` returns `null` instead of creating a new one, which is how you check "is the user logged in".

Minimal login example for a session question

```
<form action="login" method="post">
    Username: <input name="username"><br>
    Password: <input type="password" name="password"><br>
    <input type="submit" value="Login">
</form>
```

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        String u = req.getParameter("username");
        String p = req.getParameter("password");

        if ("admin".equals(u) && "1234".equals(p)) {      // or check against
MySQL
            req.getSession().setAttribute("user", u);
            res.sendRedirect("profile");
        } else {
            res.setContentType("text/html");
            res.getWriter().println("Invalid login. <a href='login.html'>Try
again</a>");
        }
    }
}
```

Writing `"admin".equals(u)` (constant first) avoids a `NullPointerException` when `u` is null.

MySQL commands to set up before running

```
mysql -u root -p          # log in, type your password
```

Then paste the `CREATE DATABASE` and `CREATE TABLE` statements shown earlier. If the table is missing, every servlet that touches the database fails with a `SQLSyntaxErrorException`.

Last-minute quick sheet

Spring Boot + Mongo, in one glance

Piece	Must contain
Model	`@Document`, `@Id String id`, `@Data @NoArgsConstructor @AllArgsConstructor`
Repository	`interface X extends MongoRepository<T, String>`
Controller	`@RestController @RequestMapping("/api/x") @CrossOrigin`, five CRUD methods
Properties	`spring.data.mongodb.uri=mongodb://localhost:27017/db`

Vue, in one glance

Piece	Must contain
main.js	`createApp`, `use(createPinia())`, `use(router)`, `mount('#app')`
Router	`createRouter({ history: createWebHistory(), routes })`
Store	`defineStore('name', { state, getters, actions })` with axios in actions
List view	`onMounted(fetchAll)`, `v-for :key`, delete button, `RouterLink` to edit

Piece	Must contain
Form view	`v-model`, `@submit.prevent`, add or update by `props.id`, `router.push('/')`

Servlet + MySQL, in one glance

Piece	Must contain
Servlet	`extends HttpServlet`, `@WebServlet("/x")`, override `doGet` / `doPost`
Output	`setContentType("text/html")`, `getWriter()`
JDBC	driver, `getConnection`, `PreparedStatement`, `executeQuery` / `executeUpdate`, close
Navigation	`sendRedirect` after insert, update, delete
State	`HttpSession`, `Cookie`, `encodeURL`, hidden field

Likely viva questions

- What is the difference between `@Controller` and `@RestController`? The second adds `@ResponseBody`, so return values become JSON.
- Why `@CrossOrigin`? Browsers block calls between different ports (5173 to 8080) unless the server allows the origin.
- `ref` vs `reactive`? `ref` wraps any value and needs `.value`; `reactive` is for objects only.
- `computed` vs `watch`? `computed` derives a value; `watch` runs side effects.
- Why Pinia over props? Shared state without passing data through every level.
- `forward` vs `sendRedirect`? Server-side, same request, URL unchanged, versus a new browser request with a changed URL.
- `doGet` vs `doPost`? GET puts data in the URL and is for reading; POST puts it in the body and is for changes.
- `Statement` vs `PreparedStatement`? Prepared is precompiled and prevents SQL injection.